ARTIFICIAL INTELLIGENCE LAB MANUAL

Submitted By:KOTA NAGA ABHINAY

Registration Number:AP23110010999

Course: Artificial Intelligence Lab

Academic Year: 2025-2026

INDEX

EXPERIMENT 1

Artificial Intelligence Problem Identification, PEAS description, and Introduction to PROLOG.

Aim: To study Artificial Intelligence concepts, identify AI problems, understand PEAS description, and get an introduction to PROLOG.

Theory

1. Introduction to Artificial Intelligence

Artificial Intelligence (AI) is a branch of computer science that focuses on creating systems capable of performing tasks that require human intelligence such as learning, reasoning, problem-solving, and decision-making.

AI systems can analyze data, recognize patterns, and make intelligent decisions. AI is widely used in fields like robotics, healthcare, gaming, and autonomous systems.

2. AI Problem Identification

An AI problem is one where:

- The solution requires intelligent reasoning
- Multiple possible solutions exist
- It can be represented using **states and actions**

Examples of AI Problems:

- Water Jug Problem
- 8 Queens Problem
- Path finding problems
- Game playing (Chess)

3. Intelligent Agents

An intelligent agent is an entity that:

- **Perceives** the environment using sensors
- **Acts** upon the environment using actuators

The goal of an agent is to achieve the best outcome.

4. PEAS Description

PEAS is used to describe an intelligent agent:

Component	Description
Performance Measure	Criteria to evaluate success
Environment	Where the agent operates
Actuators	Actions the agent performs
Sensors	Inputs received from environment

5. Example: Vacuum Cleaner Agent

Component	Description
Performance	Cleanliness, efficiency
Environment	Rooms
Actuators	Move, suck dirt
Sensors	Dirt sensor, location sensor

6. Introduction to PROLOG

PROLOG (Programming in Logic) is a logic programming language used in AI.

Key Concepts:

- **Facts** → Basic information
  Example: male(john).
- **Rules** → Logical relationships
  Example:
  father(X,Y) :- parent(X,Y), male(X).
- **Queries** → Questions asked to system
  Example:
  ?- father(john, mary).

PROLOG works using **pattern matching and logical inference**.

Result:

Thus, the concepts of Artificial Intelligence, problem identification, PEAS description, and introduction to PROLOG were studied successfully.

EXPERIMENT 2

Study of facts, objects, predicates, variables, arithmetic operators, simple input/output, and compound goals in PROLOG.

Program:

```
% Facts

student(abhi).

likes(abhi, pizza).


% Arithmetic

add(X,Y,Z):- Z is X+Y.

sub(X,Y,Z):- Z is X-Y.


% Input / Output

start :-

   write('Enter number: '), read(X),

   write('Enter number: '), read(Y),

   Z is X+Y,

   write(Z).


% Compound Goal

check(X,Y):- X>Y.

student(abhi).add(X,Y,Z):- Z is X+Y.
```

Queries:

?- student(abhi).
?- likes(abhi, pizza).
?- add(5,3,Z).
?- sub(10,4,Z).
?- start.
?- check(10,5).

Output:

```
?- student(abhi).
true.
?- likes(abhi, pizza).
true.
?- add(5,3,Z).
Z = 8.
?- sub(10,4,Z).
Z = 6.
?- start.
Enter number:
|: 5.
Enter number:
|: 3.
Sum is: 8
true.
?- check(10,5).
X is greater
true.
```

Result:

Thus, facts, predicates, variables, arithmetic operations, input/output and compound goals were successfully executed.

EXPERIMENT 3

Write a Prolog program to perform string operations (length, substring, palindrome) and set operations (union, intersection, difference).

Aim:

To implement string operations and set operations using Prolog.

Program:

```
% String Operations

str_len(Str, Len) :-
   string_length(Str, Len).

substring(Str, Sub) :-
   sub_string(Str, _, _, _, Sub).

palindrome(Str) :-
   string_chars(Str, L),
   reverse(L, L).


% Set Operations

union([], L, L).
union([H|T], L, R) :-
   member(H, L),
   union(T, L, R).
union([H|T], L, [H|R]) :-
   \+ member(H, L),
   union(T, L, R).

intersection([], _, []).
intersection([H|T], L, [H|R]) :-
   member(H, L),
   intersection(T, L, R).
intersection([_|T], L, R) :-
   intersection(T, L, R).

difference([], _, []).
difference([H|T], L, R) :-
   member(H, L),
   difference(T, L, R).
difference([H|T], L, [H|R]) :-
   \+ member(H, L),
   difference(T, L, R).
```

Output:

```
?- str_len("hello",L).
L = 5.
?- substring("hello","ell").
true.
?- palindrome("madam").
true.
?- union([1,2],[2,3],R).
R = [1,2,3].
?- intersection([1,2,3],[2,3,4],R).
R = [2,3].
?- difference([1,2,3],[2],R).
R = [1,3].
```

Result:

Thus, string operations and set operations were successfully implemented using Prolog.

EXPERIMENT 4

Develop a Prolog program to represent the family tree. Define facts and rules to find relationships such as mother, father, sibling, sister, brother, grandparent, uncle, and aunt.

Aim: To represent family relationships using facts and rules in Prolog.

Program:

```
% Facts
parent(john, mary).
parent(john, sam).
parent(lisa, mary).
parent(lisa, sam).
parent(mary, anna).

male(john).
male(sam).

female(lisa).
female(mary).
female(anna).

% Rules
mother(M, P) :-
   parent(M, P),
   female(M).

father(F, P) :-
   parent(F, P),
   male(F).

sibling(X, Y) :-
   parent(P, X),
   parent(P, Y),
   X \= Y.

sister(S, X) :-
   sibling(S, X),
   female(S).

brother(B, X) :-
   sibling(B, X),
   male(B).


grandparent(G, C) :-
```

parent(G, P),
parent(P, C).

Output:

```
?- mother(lisa, mary).
true.
?- father(john, sam).
true.
?- sibling(mary, sam).
true.
?- sister(mary, sam).
true.
?- brother(sam, mary).
true.
?- grandparent(john, anna).
true.
```

Result:

Thus, family relationships were successfully represented using Prolog.

EXPERIMENT 5

Write a Prolog program to calculate the percentage of a student and to generate the payslip of an employee using arithmetic operations.

Aim:

To implement arithmetic operations in Prolog for calculating student percentage and employee payslip.

Program:

```
% Student Percentage
student :-
   write('Enter name: '), read(Name),
   write('Enter roll number: '), read(Roll),
   write('Enter 3 subject marks: '),
   read(M1), read(M2), read(M3),
   Total is M1 + M2 + M3,
   Percentage is Total / 3,
   write('Percentage: '), write(Percentage), nl.

% Employee Payslip
employee :-
   write('Enter name: '), read(Name),
   write('Enter basic salary: '), read(Basic),
   HRA is Basic * 0.10,
   DA is Basic * 0.15,
   Gross is Basic + HRA + DA,
   write('Gross Salary: '), write(Gross), nl.
```

Output:

```
?- student.
Enter name:
|: abhi.
Enter roll number:
|: 101.
Enter 3 subject marks:
|: 80.
|: 70.
|: 90.
Percentage: 80.0
true.
?- employee.
Enter name:
|: ravi.
Enter basic salary:
|: 20000.
Gross Salary: 25000.0
true.
```

Result:

Thus, the student percentage and employee payslip were successfully calculated using Prolog.

EXPERIMENT 6

Write a Prolog program to implement recursion for finding the factorial of a number and the sum of elements in a list.

Aim:

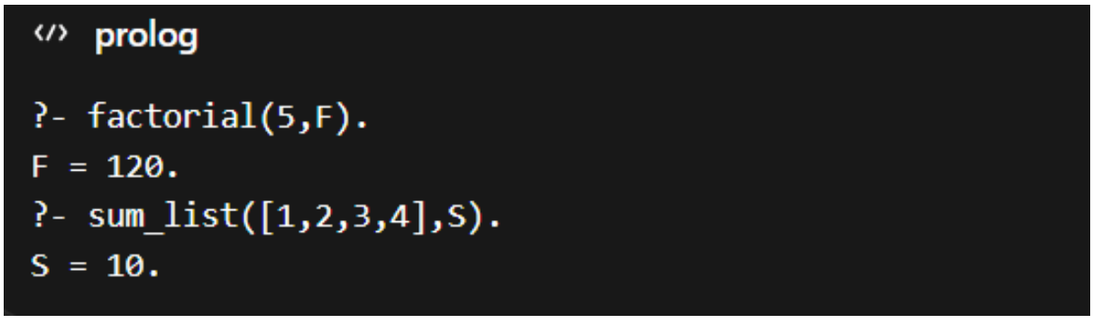
To implement recursion in Prolog for solving factorial and list sum problems.

Program:

```
% Factorial
factorial(0,1).
factorial(N,F) :-
   N > 0,
   N1 is N - 1,
   factorial(N1,F1),
   F is N * F1.

% Sum of list
sum_list([],0).
sum_list([H|T],S) :-
   sum_list(T,S1),
   S is H + S1.
```

Output:



Result:

Thus, recursion was successfully implemented in Prolog to calculate factorial and sum of list elements.

EXPERIMENT 7

Using compound object accept 10 items inventory and display. Find odd and even numbers from list.

Aim:

To implement compound objects in Prolog for inventory and to find odd and even numbers from a list.

Program:

```
% Inventory using compound object
inventory :-
   write('Enter 10 items as item(Name,Qty,Price):'), nl,
   read(I1), read(I2), read(I3), read(I4), read(I5),
   read(I6), read(I7), read(I8), read(I9), read(I10),
   display(I1), display(I2), display(I3), display(I4), display(I5),
   display(I6), display(I7), display(I8), display(I9), display(I10).

display(item(Name,Qty,Price)) :-
   write('Item: '), write(Name), nl,
   write('Quantity: '), write(Qty), nl,
   write('Price: '), write(Price), nl.

% Even numbers
even([]).
even([H|T]) :-
   0 is H mod 2,
   write(H), write(' is even'), nl,
   even(T).
even([_|T]) :-
   even(T).

% Odd numbers
odd([]).
odd([H|T]) :-
   1 is H mod 2,
   write(H), write(' is odd'), nl,
   odd(T).
odd([_|T]) :-
   odd(T).
```

Output:

```
?- inventory.
Enter 10 items as item(Name,Qty,Price):
|: item(laptop,10,50000).
|: item(mouse,20,500).
|: item(keyboard,15,1000).
|: item(monitor,5,8000).
|: item(printer,3,6000).
|: item(usb,25,300).
|: item(speaker,10,2000).
|: item(router,7,1500).
|: item(webcam,4,1200).
|: item(headset,8,1800).
Item: laptop
Quantity: 10
Price: 50000
Item: mouse
Quantity: 20
Price: 500
Item: keyboard
Quantity: 15
Price: 1000
...

?- even([1,2,3,4,5]).
2 is even
4 is even
true.

?- odd([1,2,3,4,5]).
1 is odd
3 is odd
5 is odd
true.
```

Result:

Thus, compound objects were used to implement inventory and odd/even numbers were successfully identified from a list.

EXPERIMENT 8

Write programs for the following AI problems: Water Jug Problem, Monkey Banana Problem, 8 Queens Problem, Traveling Salesman Problem, and Water Jug problem using LISP.

Aim:

To implement various Artificial Intelligence problems

Program:

1. Water Jug Problem (BFS)

```
from collections import deque

def water_jug(cap1, cap2, target):
    visited = set()
    queue = deque([(0, 0)])

    while queue:
        x, y = queue.popleft()

        if (x, y) in visited:
            continue

        visited.add((x, y))
        print("State:", x, y)

        if x == target or y == target:
            print("Target reached!")
            return

        queue.append((cap1, y))
        queue.append((x, cap2))
        queue.append((0, y))
        queue.append((x, 0))

        pour = min(x, cap2 - y)
        queue.append((x - pour, y + pour))

        pour = min(y, cap1 - x)
        queue.append((x + pour, y - pour))

water_jug(4, 3, 2)
```

Output:

```
Current state: 0 0
Current state: 4 0
Current state: 0 3
Current state: 4 3
Current state: 1 3
Current state: 3 0
Current state: 1 0
Current state: 3 3
Current state: 0 1
Current state: 4 2
Target reached!

=== Code Execution Successful ===
```

2. Monkey Banana Problem

```
def monkey_banana():
    state = ("door", "window", False, False)

    print("Initial State:", state)

    state = ("window", state[1], state[2], state[3])
    print("Move to box:", state)

    state = ("middle", "middle", state[2], state[3])
    print("Push box:", state)

    state = (state[0], state[1], True, state[3])
    print("Climb box:", state)

    if state[2] and state[0] == "middle":
        state = (state[0], state[1], state[2], True)

    print("Final State:", state)

monkey_banana()
```

Output:

```
Initial State: ('door', 'window', False, False)

Step 1: Monkey moves to box
State: ('window', 'window', False, False)
Step 2: Monkey pushes box to banana
State: ('middle', 'middle', False, False)
Step 3: Monkey climbs box
State: ('middle', 'middle', True, False)
Step 4: Monkey grabs banana
Final State: ('middle', 'middle', True, True)

Goal achieved!

=== Code Execution Successful ===
```

3. 8 Queens Problem

```
def is_safe(board, row, col):
    for i in range(col):
        if board[i] == row or abs(board[i] - row) == abs(i - col):
            return False
    return True

def solve_queens(n):
    board = [-1] * n

    def solve(col):
        if col == n:
            print("Solution:", board)
            return True

        for row in range(n):
            if is_safe(board, row, col):
                board[col] = row
                if solve(col + 1):
                    return True
                board[col] = -1
        return False

    solve(0)

solve_queens(8)
```

Output:

```
Solution: [0, 4, 7, 5, 2, 6, 1, 3]

=== Code Execution Successful ===
```

4. Traveling Salesman Problem

```
from itertools import permutations

def tsp(graph):
    n = len(graph)
    cities = list(range(n))
    min_cost = float('inf')
    best_path = []

    for perm in permutations(cities[1:]):
        path = [0] + list(perm) + [0]
        cost = 0

        for i in range(len(path) - 1):
            cost += graph[path[i]][path[i+1]]

        if cost < min_cost:
            min_cost = cost
            best_path = path

    print("Minimum cost:", min_cost)
    print("Best path:", best_path)

graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]

tsp(graph)
```

Output:

```
Minimum cost: 80
Best path: [0, 1, 3, 2, 0]

=== Code Execution Successful ===
```

5. Water Jug (LISP Style / Recursion)

```
def water_jug(x, y, visited):
    if x == 2 or y == 2:
        print("Goal reached:", (x, y))
        return True

    if (x, y) in visited:
        return False

    visited.append((x, y))

    if water_jug(4, y, visited):
        print("Step:", (4, y))
        return True

    if water_jug(x, 3, visited):
        print("Step:", (x, 3))
        return True

    if water_jug(0, y, visited):
        print("Step:", (0, y))
        return True

    if water_jug(x, 0, visited):
        print("Step:", (x, 0))
        return True

    pour = min(x, 3 - y)
    if water_jug(x - pour, y + pour, visited):
        print("Step:", (x - pour, y + pour))
        return True

    pour = min(y, 4 - x)
    if water_jug(x + pour, y - pour, visited):
        print("Step:", (x + pour, y - pour))
        return True

    return False

water_jug(0, 0, [])
```

Output:

```
Goal reached: (4, 2)
Step: (4, 2)
Step: (3, 3)
Step: (3, 0)
Step: (0, 3)
Step: (4, 3)
Step: (4, 0)

=== Code Execution Successful ===
```

Result:

Thus, various AI problems were successfully implemented